5(a) . IMPLEMENT A* SEARCH ALGORITHM

```
import heapq

def astar(graph, heuristic, start, goal):

    if start not in graph:
        print("Invalid start node")
        return

    if goal not in graph:
        print("No path exists")
        return

    pq = [(heuristic[start], start)]

    g = {start: 0}
    parent = {start: None}
    visited = set()

    while pq:

        f, current = heapq.heappop(pq)

        if current in visited:
            continue

        visited.add(current)

        if current == goal:
```

```
            path = []
            node = goal

            while node is not None:
                path.append(node)
                node = parent[node]

            path.reverse()

            print("Path:", " -> ".join(path))
            print("Total Cost:", g[goal])
            return

        for neighbour, cost in graph[current].items():

            new_g = g[current] + cost

            if neighbour not in g or new_g < g[neighbour]:

                g[neighbour] = new_g
                parent[neighbour] = current

                new_f = new_g + heuristic[neighbour]

                heapq.heappush(pq, (new_f, neighbour))

    print("No path exists")



# Read graph
graph = {}
```

```
n = int(input("Enter number of nodes: "))

for i in range(n):

    node = input("Enter node: ").strip()

    graph[node] = {}

    e = int(input("Enter number of neighbours: "))

    for j in range(e):

        neighbour, cost = input(
            "Enter neighbour and cost: "
        ).split()

        graph[node][neighbour] = int(cost)



# Read heuristic values
heuristic = {}

print("\nEnter heuristic values:")

for node in graph:

    heuristic[node] = int(
        input("h(" + node + ") = ")
    )
```

```
# Multiple start and goal inputs
while True:

    print("\n--- A* SEARCH ---")

    start = input("Enter start node: ").strip()
    goal = input("Enter goal node: ").strip()

    astar(graph, heuristic, start, goal)

    choice = input(
        "\nDo you want to try again? (yes/no): "
    ).strip().lower()

    if choice != "yes":
        print("Program terminated.")
        break
```

OUTPUT:

Test case 1 – Find the optimal path

Test case 2 – Start node is the goal node

Test case 3 – Find a path to an intermediate node

Test case 4 – Find a path through another branch

Test case 5 – Goal node does not exist

Test case 7 – Direct path versus alternative path

Test case 8 – Invalid start node

IDLE Shell 3.13.5

File Edit Shell Debug Options Window Help

```
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
============== RESTART: C:/Users/BMSCE/Documents/AI lab/a search.py ==============
Enter number of nodes: 8
Enter node: A
Enter number of neighbours: 3
Enter neighbour and cost: B 1
Enter neighbour and cost: C 2
Enter neighbour and cost: H 7
Enter node: B
Enter number of neighbours: 2
Enter neighbour and cost: D 4
Enter neighbour and cost: E 6
Enter node: C
Enter number of neighbours: 2
Enter neighbour and cost: F 3
Enter neighbour and cost: G 2
Enter node: D
Enter number of neighbours: 1
Enter neighbour and cost: E 7
Enter node: E
Enter number of neighbours: 0
Enter node: F
Enter number of neighbours: 1
Enter neighbour and cost: H 1
Enter node: G
Enter number of neighbours: 1
Enter neighbour and cost: H 2
Enter node: H
Enter number of neighbours: 0

Enter heuristic values:
h(A) = 5
h(B) = 3
h(C) = 4
h(D) = 2
h(E) = 6
h(F) = 3
```

Ln: 97 Col: 0

IDLE Shell 3.13.5

File Edit Shell Debug Options Window Help

```
h(G) = 1
h(H) = 0

--- A* SEARCH ---
Enter start node: A
Enter goal node: H
Path: A -> C -> G -> H
Total Cost: 6

Do you want to try again? (yes/no): YES

--- A* SEARCH ---
Enter start node: A
Enter goal node: A
Path: A
Total Cost: 0

Do you want to try again? (yes/no): YES

--- A* SEARCH ---
Enter start node: A
Enter goal node: D
Path: A -> B -> D
Total Cost: 5

Do you want to try again? (yes/no): YES

--- A* SEARCH ---
Enter start node: A
Enter goal node: F
Path: A -> C -> F
Total Cost: 5

Do you want to try again? (yes/no): YES

--- A* SEARCH ---
Enter start node: A
Enter goal node: Z
No path exists
```

Ln: 97 Col: 0

```
Do you want to try again? (yes/no): YES

--- A* SEARCH ---
Enter start node: A
Enter goal node: H
Path: A -> C -> G -> H
Total Cost: 6

Do you want to try again? (yes/no): YES

--- A* SEARCH ---
Enter start node: X
Enter goal node: H
Invalid start node

Do you want to try again? (yes/no): NO
Program terminated.
```

Ln: 97 Col: 0

Test case 6 – Goal node is disconnected

```
============================== RESTART: C:/Users/BMSCE/Documents/AI lab/a search.
py ==============================
Enter number of nodes: 3
Enter node: A
Enter number of neighbours: 1
Enter neighbour and cost: B 2
Enter node: B
Enter number of neighbours: 1
Enter neighbour and cost: A 2
Enter node: C
Enter number of neighbours: 0

Enter heuristic values:
h(A) = 2
h(B) = 1
h(C) = 0

--- A* SEARCH ---
Enter start node: A
Enter goal node: C
Goal node is disconnected

Do you want to try again? (yes/no):
```

Ln: 47 Col: 36